2.1 Overview of Existing Systems

Modern platforms in the gig economy such as Upwork, Fiverr, and Ethlance are very centralized and offer a great number of features such as project bidding, escrow, reputation scoring and more. However, these systems come with a whole host of problems, including high fees for services, lack of transparency and data ownership issues. On the other hand, some of these problems are already addressed by decentralized platforms such as Ethlance.

The platform uses smart contracts and decentralized storage which appears to be far more complicated in terms of usability, scalability, and adoption by the user due to complex interfaces and high transaction costs. Adoption of these systems is heavily use-case driven from the frontend. Studies have indicated that a lack of proper UI/UX design might be one of the most significant obstacles to mass adoption of Web3 platforms. (Ethereum_Whitepaper_-_Buterin_2014)

2.1.1 Existing System Summaries

This section will showcase multiple research papers, their reviews, and undertakings in the freelancing platforms, decentralized systems, and modern web technologies. It is an attempt to know what previous studies have utilized in respect to a freelancing ecosystem - specifically on System Interactions.

2.1.2 Shahzad (2024) - Security and Platform Vulnerabilities

Shahzad, in 2024, took a close look into security challenges that centralized freelancing platforms faced and pointed out risks connected with data breaches, identity theft, and lack of transparency. The study claimed that most centralized systems face challenges related to breaching, transparency, and poor identity management. The author tried to capture the perception and usage of digital platforms by users using models like the Technology Acceptance Model.

The study integrates theoretical models, including the Technology Acceptance Model and the Information Processing Theory, into the understanding of user interaction with digital systems. One key point highlighted in this paper is that security is not just a backend issue but largely influenced by the UI/UX. Poorly designed interfaces thus expose users to phishing attacks, unclear authentication flows, and insecure interactions.

This suggests:

- Present the interfaces of authentication in a clear manner.
- Password feedback correctly done in login procedures
- Minimize chances of user action confusion on such sensitive actions.

Relevance to this Project:

Frontend design in the proposed system integrates wallet-based authentication and the associated UI prompts required for clear guidance on secure interaction flows. It is the job of the frontend to warn the users about what they are signing and how not to make accidental transactions. This paper extends the point of security privacy onto the interface saying it matters as much as it does on the backend for an interactive system. If the interface is unclear, users may not understand what actions they are performing, especially during login or authentication.

The paper suggests that systems should provide clear and friendly authentication processes. This directly relates to my project, where MetaMask wallet login is used. In this case, the frontend must clearly guide users during wallet connection and signature approval to avoid confusion.

2.1.3 Kunwar et al. (2024) - Job Matching Algorithm

Kunwar et al. (2024) concern the enhancement of the freelancer-client matching based on the rule-based and algorithmic solutions. The study assesses the possibility of using structured information (i.e. skills, experience, and availability) to improve the accuracy of recommendations.

As much as the study is more of a backend, it has an indirect effect on frontend design. Recommendations should be visually clear and filters user-friendly to have efficient matching systems.

Key contributions include:

- Rule-based matching systems
- Better job accuracy.
- Systematized data use.

Frontend implications:

- Requirement of interactive dashboards.
- Sort and filter functionality.
- Well-presented matched results.

Significance to this project:

The frontend should show recommendations in a friendly interface, with React components to get the job listings dynamically updated with the responses of the backend. To properly display the results from the matching system on the interface, all these front-end issues must be gotten right. Bad front-end design may lead to a situation whereby users do not get to realize the full

benefits of the enhanced matching system as laid out in the functionalities delivered by the back-end team.

2.1.4 Singh et al. (2024) – DLancer: Blockchain-Based Freelancing Platform

Singh et al. (2024) came up with a decentralized freelancing platform built on Ethereum smart contracts and designed by IPFS, aiming at providing transparency, immutability, and secure payment mechanisms.

The major limitation underscored in the study is poor user interface and high user complexity in interaction. These were pointed out as the key challenges:

- Complex transaction processes
- Lack of an intuitive UI
- High dependence on technical knowledge

Frontend perspective:

The following points were highlighted in this paper:

- Making blockchain interactions easy
- To make transparent the status of the transactions
- The user interface should be very friendly towards a non-technical user.

Significance to this project:

The system is an improvement over DLancer in that it includes a React-based front end, abstracting away the complexity of blockchain from the users to ensure a seamless experience. Clearly, a frontend view of this study provides for a good development perspective, but clumsy UI design would wholly negate it. In my project, that is solved by designing a simple and clean interface with React, where blockchain interactions are much reduced and thereby ease of use for the user.

2.1.5 Gupta et al. (2020) – Challenges in Software Development with Freelancers

Gupta et al. (2020) talk about freelance software development environments. Challenges affect majorly the areas of collaboration, communication, and coordination.

It appears that:

- 33% of problems are related to coordination.

- UI is implicated in effective communication.
- Poor interface design breeds inefficiency

Frontend Relevance:

- The UI should permit collaborative features.
- The interfaces should lower the cognitive load.
- Clear navigation will enhance productivity.

Relevance to this Project:

Frontend design should provide:

- Easy tracking of projects
- Clear communication of the status of projects
- Simple navigation between features.

2.1.6 Tamsetwar et al. (2026) – Hybrid Web3 Architecture for Decentralized Gig

In 2026, Tamsetwar et al. proposed a hybrid Web3 which combines partly decentralized with partly centralized structures in one architecture. Decentralized platforms suffer from issues related to scalability and performance.

This includes:

- Usability nightmares in fully decentralized systems.
- Hybrid systems increase performance.
- UI complexity is a major hurdle.

Frontend insights:

- Intuitive UI for complex systems
- Responsive and fast interfaces are very essential.

- Elaboration of blockchain interactions

How this fits with the project:

The project uses a mixed architecture with front end, letting users confront it without being disturbed. It has been adapted to hybrid architecture. A key takeaway from this research is that fully decentralized systems are mostly excellent but have low user experience. The interaction between users is hard due to the system of blockchain-based systems being distributed.

2.1.7. Fulker & Riedl (2024) - Cooperation & Coordination in Gig Economy

In their research conducted in 2024, Fulker and Riedl established the importance of communication and coordination on such freelancing websites. It is an important aspect of front-end design because an interactive interface is engaging to the user. This may be improved by clear project updates, user profiles, and those of activities tracking to enhance collaboration.

It can be assumed that most freelancers prefer working together rather than alone.

However, good communication makes all operations on the site to be carried forward and completed in time, to the satisfaction of the client. Platforms that do not offer collaboration features do not encourage users' engagement in them.

Frontend Relevance:

The UI should be effective for satisfactory communication and interaction. It should work in such a way that it should bring collaboration among users naturally. The display has to be clean so that all activities and updates by the user are clear.

The front end should not only serve the information but also try to create an environment where the user feels comfortable with the system. Interactive and responsive UI elements are one of the ideas to be considered in this project.

2.1.8. Buterin (2024) - Ethlance; Decentralized Freelancing Platform

Ethlance is a blockchain-based freelancing platform where intermediaries are not kept anymore, and there can be direct communication between the freelancers and their clients.

Looks like:

- Fully decentralized platform with no middleman
- Users have full control over their data
- Transactions are recorded on the blockchain

Frontend Relevance:

- The UI should be able to simplify the complexity of a blockchain.
- Clear navigation supports usability.
- User onboarding should be straightforward.

Relevance to this Project:

Frontend design should serve to achieve:

- Have a simple onboarding process.
- Easy understanding of system features.
- Improved GUI in comparison to currently existing platforms.

2.1.9. Kasenda et al., (2024) - Role and Evolution of Frontend Developers in the Industry

Frontend developers have evolved from simple web designers to full stack engineers out of necessity. The frontend field itself has undergone significant transformations moving from simple HTML pages to more complex modern frameworks such as React, Vue, or Angular.

Frontend Relevance:

Current frontend components; particularly; React or React.js allows for teams to build and manage system interfaces whilst maintaining performance, accessibility standards, testing and DevOps.

The frontend should be team focused, with automated testing and streamlines development workflow.

The frontend should also be able to simply bridge the complex backend systems with intuitive user interfaces.

2.1.10 Reuter et al., (2022) - Comparative Analysis on Various CSS and JavaScript Frameworks

In the research conducted, three popular CSS frameworks i.e. Bootstrap, Tailwind CSS and Material UI for React were compared based off of performance metrics such as the time taken to load instances, the total page size loaded, the total design aesthetics, the ease of use of the various frameworks, response time, responsiveness, and learning curve.

Key Findings:

- Tailwind CSS has the best performance with fastest time to load but smallest page size.
- Bootstrap has the best documentation and community support.
- Material UI has the best design aesthetics suited for modern designers.

Frontend Relevance

Custom CSS gives developers full control over styling, it allows for cleaner as well as more optimized code tailored for specific projects. However, coding in custom CSS requires strict adherence to coding conventions as well as constant monitoring for various components and their architectures.

This project should implement custom CSS to ensure optimal performance, avoid overhead from framework utilities and a smoother experience for users interacting with the system.

2.2 Overall Findings and Analysis

All the above papers' reviews give an array of key points.

First of all, most systems concentrate on backend technologies like blockchain, smart contracts, data storage, among others, but tend to ignore frontend design. The result is a large number of platforms that are hard to use.

Secondly, one of the essential issues with decentralized systems is usability. In case the interface is not properly set, the user might get confused with the features of the blockchain.

Finally, one very clear gap in research is that of having intuitive interfaces for Web3. Most research only covers the technical improvements and does not concentrate on user experience.

2.3.1 Methodology Analysis

A glance at freelancing platforms and freelancers reveals that a lot of them are realized using Hybrid architecture model, System design methodology, and Modular development approach.

a. Systematic Design Approach

Most of the works, including that of Singh et al. 2024 and Tamsetwar et al. 2026, have systematically designed the architecture in layers, such as:

- Frontend (User Interface)
- Backend (API and logic) re-enabling smart contracts on
- Blockchain Storage (Database/IPFS)

It is user-friendly in terms of frontend as it allows developers to just concentrate on the user interface and interaction between other components by APIs.

In the project, it adopts a similar kind of approach where the frontend has been independently developed by React, which interacts with the backend services and blockchain components.

b. Development by Components

Component-based architecture is definitely an important point of modern front-end development, working perfectly with such frameworks as React. This approach breaks down an interface into reusable components that contain:

- Navigation bar
- Project cards
- User profile sections
- Forms and dashboards

The above-cited methodology is expected to enhance:

- Code Reusability
- Time Reduction in Maintenance
- Scalability

It sounds to be one of the best ways to develop large-volume applications effectively. The React components will help users manage different sections of the UI in this project to simplify location and updating when required.

c. Hybrid Web3 Development Approach

In most cases, research has suggested that truly decentralized systems come with usability and performance bottlenecks. A combination of centralized backends and decentralized components, including a blockchain and IPFS, most likely underlies current systems in use. The method also brings down the system complexity for better performance and user experience.

This methodology helps in:

- Reducing system complexity
- Improving performance
- Enhancing user experience

In terms of frontend point of view, the User Interface can be kept fast and responsive while still connecting with decentralized systems.

2.4 Summary of Key Findings and Research Gaps

2.4.1 Key Findings

Important findings emerged out of several research papers analyzed. Firstly, most centralized systems leveraged by traditional freelancing platforms are better in performance and more user-friendly from the interface's perspective; still, they have been plagued with issues such as high service fees, lack of transparency, and limited user control over data, therefore reducing the level of trust between a client and a freelancer.

Besides, decentralized platforms built on blockchain technology ensure more transparency, security, and trust in the presence of smart contracts and distributed storage systems such as IPFS. These systems eradicate any intermediaries and offer enhanced control over data from users. Unfortunately, in most cases, they add more problems of complexity regarding the usage and the interaction. Another key finding is that usability plays a significant role in the success of freelancing platforms. Many have been inclined to pieces of research showing users' preference for simplicity of design, responsiveness, and ease of interaction within the user interface. Poor interface design can lead to a negative experience on the part of users; this is even more critical in Web3 systems where users must deal with wallets and blockchain transactions. Hybrid architectures that combine centralized and decentralized components were found to be more practical.

2.4.2 Identified Research Gaps

Although some research gaps have been identified through the analysis of literature, probably the biggest oversight has been the lack of attention given to the front end of development in a decentralized system. Most emphasize back-end technologies, like blockchain, smart contracts, and storage mechanism affecting the importance on user interface design. Another gap is fewer in number of research in making Web3 interfaces more "user-friendly." Web3 platforms, on the other hand, almost always turn out not to be user-friendly because they require the users to have some base knowledge in technically oriented concepts such as wallets, private keys, and transaction confirmations. Also, it's nearly impossible to effectively integrate between the front-end and back-end systems. Even if the back end has a solution for finalization, a bad front end will somehow manage to screw up the system's efficiency and user-friendliness. Moreover, not enough work has concentrated on making a responsive, interactive UI design that would foster better user engagement and overall system usability.

2.4.3 Connection to the Current Project

The project will seal the mentioned gaps concentrating on technical and user experience.

In this project, it is a frontend that is being developed with the help of React, JavaScript, HTML, and CSS so that the user could comfortably operate within the system. Specific attention has been paid especially in rendering intricate blockchain interaction processes such as wallet verification and transaction functionalities simple through unambiguous and straightforward interface designs.

It does the project by taking hybrid architecture where centralized back-end services are coupled with decentralized ones such as blockchain and IPFS. This will make the system operate at full efficiency to ensure its transparency and security.

This project helps remove the divide between fine advanced decentralized technologies and friendliness of design to make the platform more practical and accessible to users by succeeding in paying attention to frontend usability, without focusing on the system functionality.

2.5 Summary table of past work

Author	Yr	Focus Area	Methodology	Technology/Tools	Key Findings	Limitations
Shahzad	2024	Security in freelancing platforms	Theoretical models (TAM, IPT)	Centralized systems	Identified vulnerabilities in existing platforms	Limited focus on UI/UX design.
Kunwar et al.	2024	Job matching systems	Rule-based algorithm	Data-driven systems	Improved freelancer-job matching accuracy	No focus on frontend presentation

Author	Yr	Focus Area	Methodology	Technology/Tools	Key Findings	Limitations
Singh et al.	2024	Decentralized freelancing (DLancer)	Blockchain-based architecture	Ethereum, IPFS	Transparent and secure transactions	Complex UI, poor usability
Gupta et al.	2020	Freelance software development challenges	Analytical study	Collaboration systems	It found collaboration and coordination problems	Limited technical UI solutions
Tamsetwar et al.	2026	Hybrid Web3 architecture	Hybrid system design	Blockchain + centralized DB	Offers better scalability and performance	Still suffering from UI complexity
Fulker & Riedl	2024	Collaboration in freelancing	User behavior analysis	Social platforms	Underlined the importance of collaboration	Social platforms But lacking in system-level implementation
Kasen da et al.	2024	Role & Evolution of Frontend Developers	Industry Analysis	React, Vue, Angular	Explores different frameworks, and their characteristics	Limited practical implementation; only guidelines
Reuter et al.	2022	Comparative analysis of CSS frameworks	Comparative analysis	Bootstrap, Tailwind, Material UI	Tailwind best performance; Bootstrap best documentation; Material UI best aesthetics	Only 3 frameworks analyzed

2.6 Individual Analysis Statement

Most of the existing research has been directed at making technical freelancing platforms better. Older work conceptualized developments in security, blockchain integration, and job matching algorithms. The contribution is an incredibly crucial part which mostly neglects the role of front-end development in ensuring that systems are usable and satisfactory to users.

Most decentralized platforms are filled with complex interfaces and bad interaction designs that often bring challenges to users. For this, such a reduction is realized in systems that otherwise might have been termed advanced. I would, therefore, argue that the gap between complex back-end technologies and the end user can be bridged by the front-end development.

I happen to be the front-end developer on this project with respect to the user-friendly responsive interface design using React, JavaScript, HTML, and CSS for users to interact with the system very easily at the time of wallet authentication or transaction processes based on blockchain.

The thing I learned is component-based architecture to be used when developing in React, it was quite useful in developing a scalable and maintainable system. By decomposing the interface into few reusable parts, it begins to have a well-organized style and is simple to handle.

In conclusion, this project will work to combine effective back-end technologies and good front-end design to ensure that not only is the system technologically advanced, but also that it is practical and easy to use by the user.

Chapter 3

System Analysis

Theoretical Analysis - SWOT Study

In this section, based on the technologies used in the proposed system, SWOT is used to cover front-end development done using React, JavaScript, HTML, and CSS.

3.1 Technical Analysis

This section will further discuss the technicalities of the system, most especially those directed towards the front-end development. It includes analysis of system components, their mechanisms of interaction, and tools used in implementing the user interface.

3.1.1 Tools Analysis

Besides the tech side, there are various tools for supporting frontend development. Choice of the right technologies is very important in fueling an effective system that is user-friendly. In this project, the front end was built up with React.js, JavaScript, HTML, and CSS; implementations have made sure these four are very standard languages currently used in most development processes.

Introduction of Frontend Development Tools

a. Code Editors (VS Code) The most common means of writing and managing code is Visual Studio Code, which provides the following:

- Editing of syntax
- Extending for React and JavaScript
- Debugging tools for this reason helps to develop at speed and be more efficient.

b. Browser Developer Tools Browser tools are useful for:

- Debugging UI issues
- Responsive Testing
- Inspection of Elements

Basically, these front-end development tools help debug errors.

c. Version Control (Git/GitHub)

The version control systems look after the changes done in the code, collaborative work with the team, or the version management of the project. This becomes very important when working on group projects for proper coordination and updates.

d. API Testing Tools

Postman is one of the tools used for the testing of the backend APIs. It is related to backend, but even frontend developers use it in order to:

- Know the API responses
- Test the data flow

3.2.1 Frontend Architecture and Design

The front end of the system is designed in React with component-based architecture. As per this approach, the face of the user interface shows itself through small components that entail navigation bars, project cards, forms, and dashboards. This architecture will provide the following benefits:

• Better organization of code

• Reuse of code between components

• Easily maintainable and updated

In my opinion, it is a very good approach for an application at a large scale because this way, all parts can be managed in isolation by the developers.

3.2.2 User Interaction and System Integration

The front end connects to the back end via APIs and communication to the blockchain sections using JavaScript. It must record information on the user to invoke API calls on presentation of information, use of forms and interoperative with wallet systems such as MetaMask on blockchain-based applications and therefore fuel UI changes as the system reacts dynamically. In this respect, JavaScript is instrumental in managing asynchronous processes and providing a smooth flow of communication between various components of a system. To enhance the user experience, special focus has been on the simplification of complex interactions in this project especially interactions involving blockchain transactions.

3.3. SWOT Analysis

React is a strong frontend framework that can be used to develop scalable and performance-based applications. Although it is a bit complex and requires the use of other libraries, the high community backing and the demand in web development make it a favorite in current web development also with lot of job opportunities here. On behalf of my team, react has been taught to us from past few years and it is easier for us to comprehend and work on.

A. React

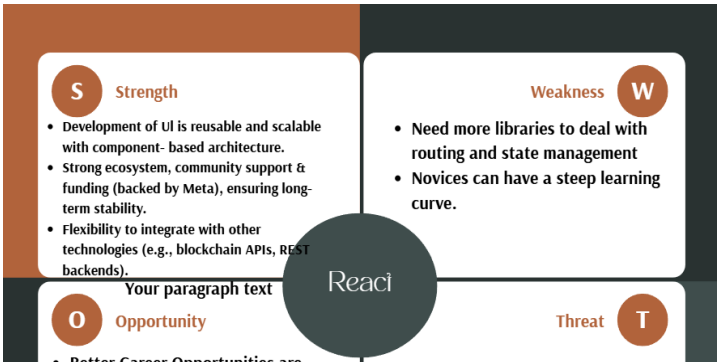


Fig 1: SWOT analysis for React framework

B. VUE

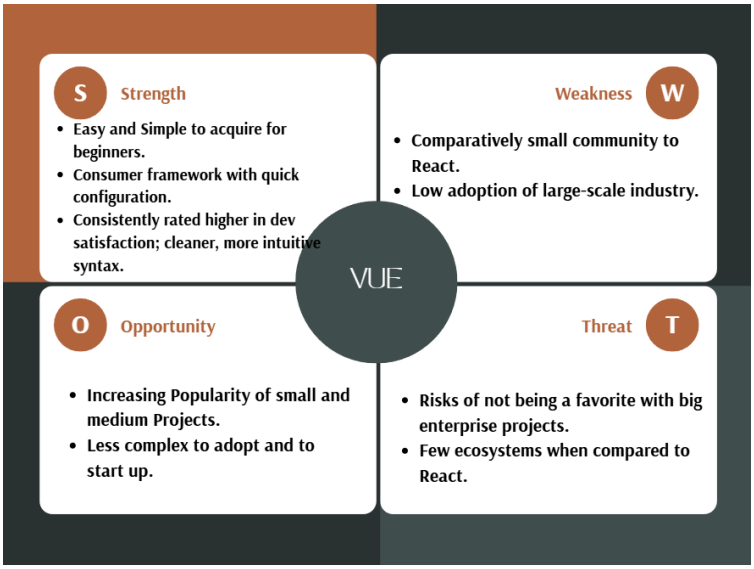


Fig 2: SWOT Analysis of Vue

Hence, React is considered better framework compared to Vue for our platform as it provides appropriate teamwork opportunities, flexibility and scalability.

C. Figma

Figma has been taught to us from very starting period of college for UI/UX design. The benefits of Figma became clear within the early phases of the project. The collaborative environments alongside the design inspection tools proved invaluable. The tools allowed the frontend and backend developers to know the structure and interface of the system, and ultimately increased the team's output tremendously. Since everyone in the team was familiar with figma it was easier to make changes, collaborate with backend and also amend to new prototype.

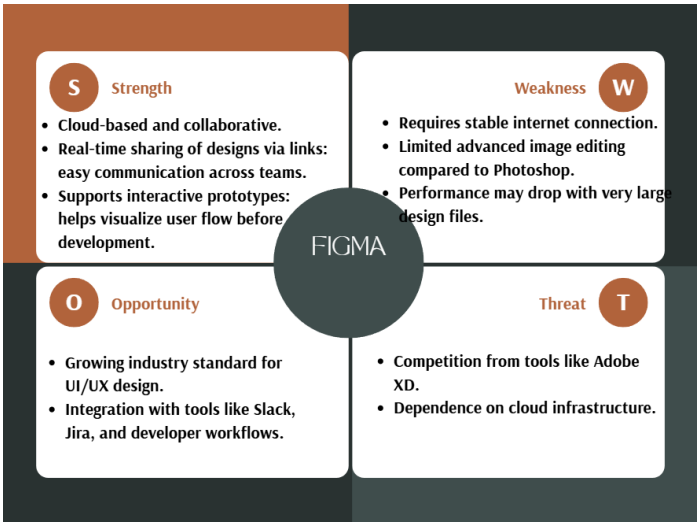
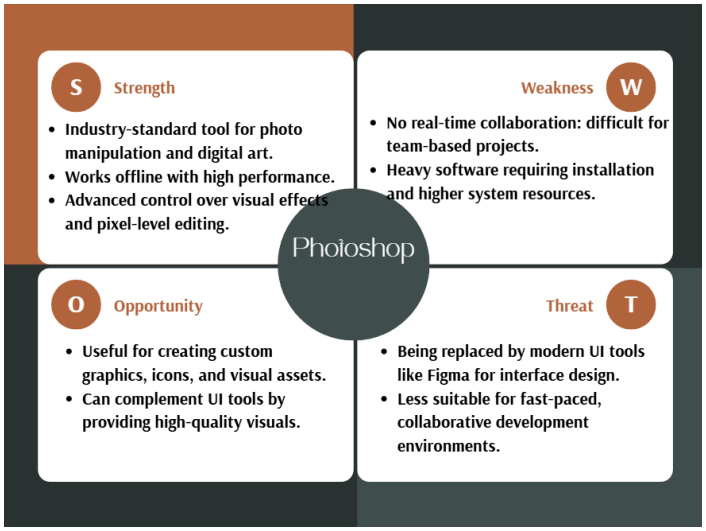


Fig 3: SWOT analysis for Figma

D. Photoshop



Since, photoshop lacks real-time collaboration and limited UI/UX workflow support it was discarded. Also vast amount of frondend & backend developers are not familiar with photoshop due to its absence of adaptability to iterative designing environment.

3.3.1 Project Tool Selection

React is highly scalable and flexible, thanks to its component-based architecture and a large ecosystem of libraries and tools. The framework's strong industry adoption and backing from Meta guarantee future development and stability. Furthermore, React allows application architecture to be tailored to project specifications, even for complex systems. React also works with contemporary technologies like REST APIs and blockchain libraries, making it a solid fit for decentralized applications, such as Freeledger. Additionally, a wider number of libraries is

readily available. React is industry and community-tested, and has the potential to be integrated with the blockchain APIs, among other modern technologies. In conclusion, Even though Vue offers various, beneficial, less complex, and easy to learn options, simplicity was disregarded for the scoring system to be more effective, configurable, adaptable, and fore-sighted.

On the other hand with strong thought for flexibility and collaboration, Figma was chosen as the primary design tool for the project. Figma allows the team to design, test, and iterate prototypes quickly, and supports the needs of modern full-stack environments. For example, as opposed to Photoshop, where image editing resources are heavily prioritized, Figma allows real-time collaboration and participation from both the frontend and backend developers. Figma also improves communication, decreases design/development discrepancies, and maintains design consistency to the final outcome. Figma allows for quick and easy access to inspection tools to pull design specifications such as spacing, format, colors and structures of components, aiding both backend and frontend developers. Overall, Figma improves team design and development workflow, and allows for seamless integration of the processes.

3.4 Requirement Specifications.

This part will include the requirements of the Freeledger system, both functional and non-functional requirements, on the front end.

3.4.1Functional Requirements

The front end is supposed to be functional as follows:

1. *User Management*
    - Connect wallet -MetaMask authorization.
    - User Profile Information View.
    - Handle session state
2. *Project Management*
    - Butt-kick a new freelance project.
    - Examine online employment opportunities.
    - Apply for the project
    - Monitor the project progress.
3. *Interaction & Communication*
    - Display project requirement information.
    - Show achievements and position.
    - Success and error messages: User response to actions.
4. *File Handling*
    - Upload deliverables to IPFS
    - Retrieve the uploaded contents to be displayed.

3.4.2 Non-Functional Requirements.

1. *Performance*

- Page load will be quick- less than three seconds.
- UI components should be rendered in a very efficient manner.

2. *Usability*
    - Simple and easy navigation
    - Extremely brief learning curve by novice users.
    - Clear visual hierarchy
3. *Scalability*
    - Elevating simultaneous use by more than one user.
    - Further extension: Modular design.
4. *Security*
    - intelligible communication with backend APIs.
    - Secure process of user input (validation)
    - Secured access to typical attacks (XSS, CSRF)
5. *Reliability*
    - Consistent system behavior
    - Proper recovery and error handling.
6. *Maintainability*
    - Clear and formalized style of coding.
    - Easily debuggable and updatable.

3.5 Resource Requirements

The success of the system is directly dependent on numerous resources such as hardware, software, and human.

a. Hardware Requirements

- Personal computer or laptop
- Good internet connectivity

b. Software Requirements

- React.js
- JavaScript, HTML, CSS
- Code editor (VS Code)
- Browser: Chrome, Firefox
- VCS (GitHub)

c. Human Resources

- Frontend Developer
- Backend Developer
- Blockchain Developer

- System Designer

3.6 Monitoring and Control

Monitoring and control are required to keep the project running according to the plan and to meet its mission objectives.

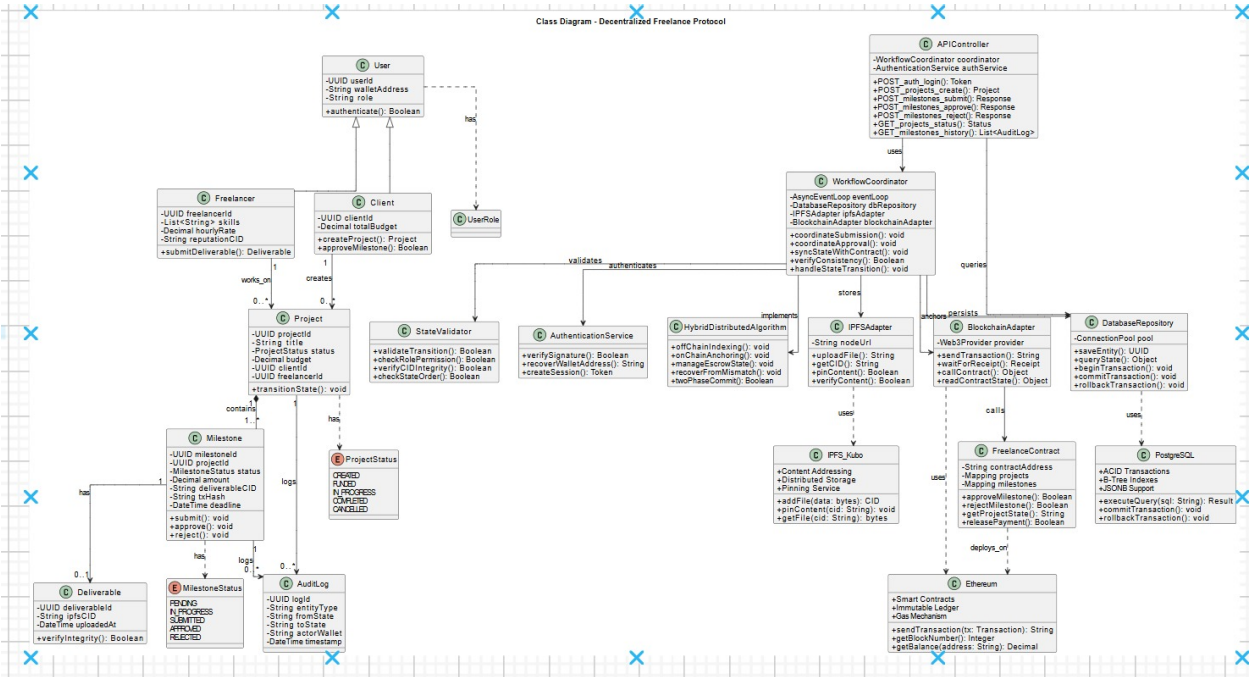
- Also, a follow-up system will be kept up for the development tasks and the milestones.
- Version Control: Manage your changes in code using GitHub and collaborate.
- Testing and Debugging: Keep checking continuously with the browser tools and API-testing tools.
- Performance Monitoring: Ensure the speed of the system, its responsiveness, and error handling.
- Feedback Mechanism: Collected feedback from users and implemented that feedback to further improve the system.

4.1 System Analysis

4.1.1. Class diagram

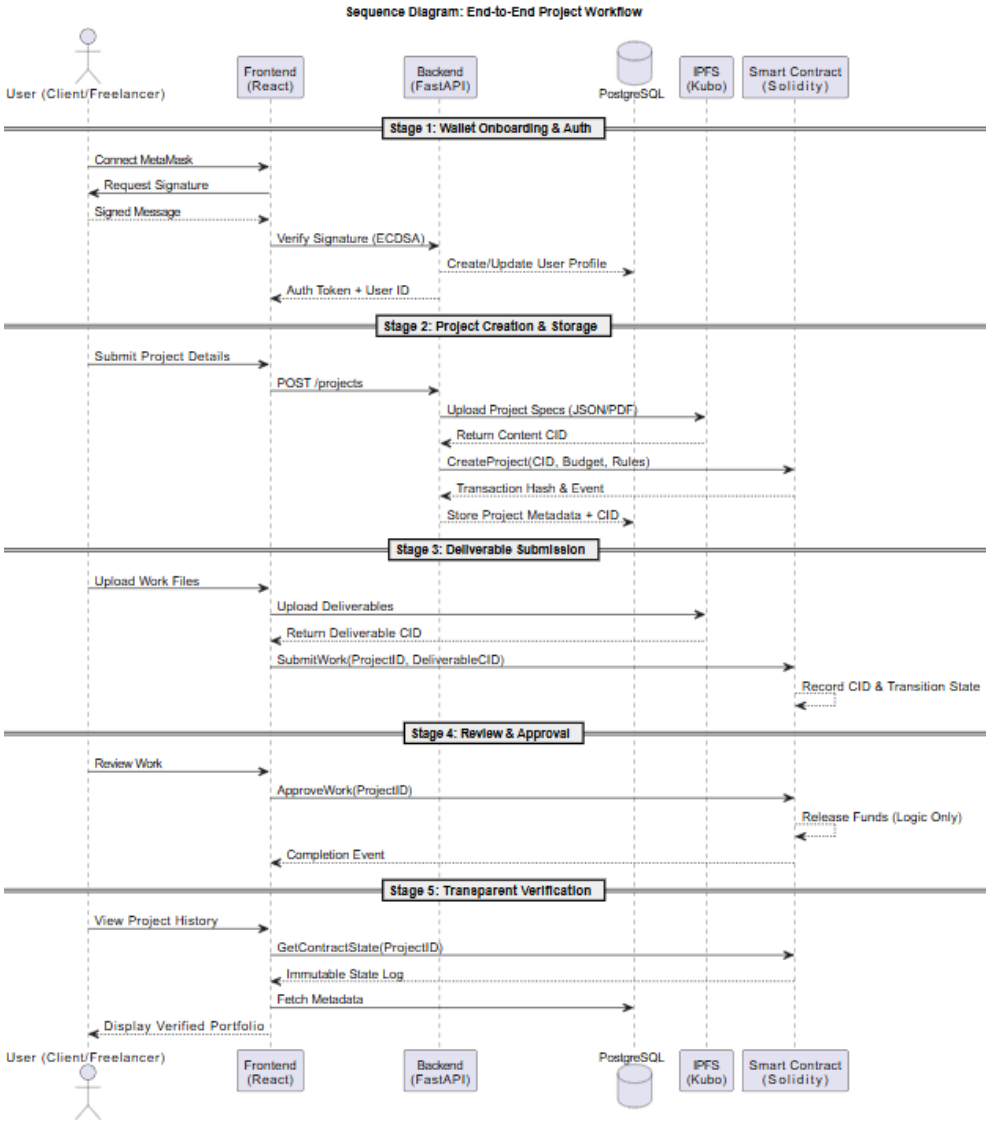
The class diagram describes about the main data structures and service architecture of the system. This includes domain entities such as User (Client and Freelancer), Project, Milestone, and Deliverable, and their relationships. For service components, the Workflow Coordinator, Authentication Service, IPFS Adapter, Blockchain Adapter, and Database Repository implement

the business logic and external integrations. In frontend development, the class diagram is important for the understanding of data models and the API contracts. It describes the organization of data storage in state (i.e., of React state management), how data is rendered in UI components, and provides for consistency between the frontend and the backend, especially with respect to project states, milestones, and blockchain updates.



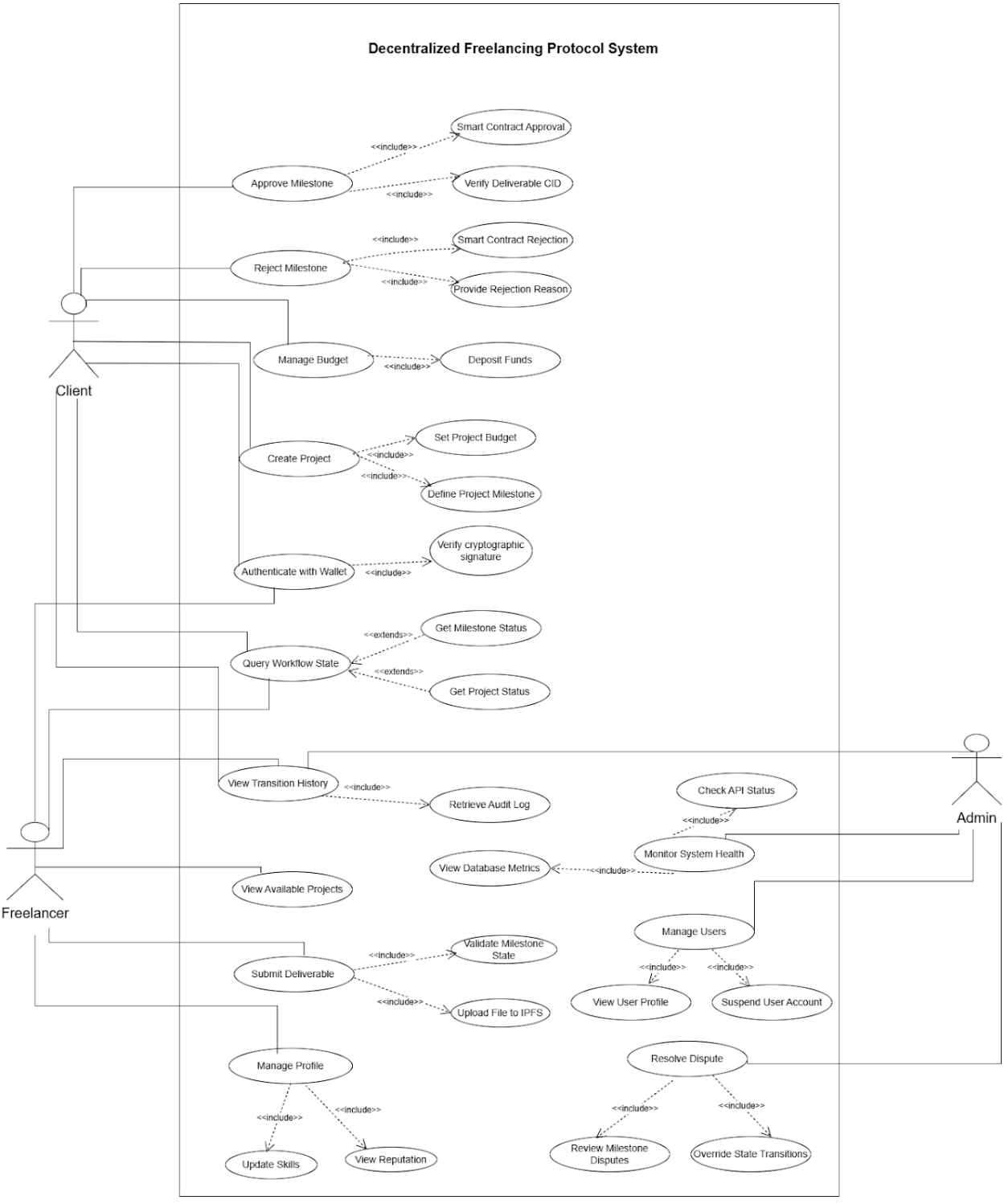
4.1.2. Sequence diagram

The front end, back end, IPFS, databases, and blockchain are all represented in the sequence diagram. It begins with authentication via a crypto wallet, where the front end parts with a Web3 provider and initiates a request for a signed message, which gets confirmed by the back end. While creating a project, the front end collects input and sends it through API calls; the back end then facilitates IPFS file uploads and deploys or interacts with a smart contract to register the project on the blockchain. While submiting deliverables, the front end handles file uploads and initiates back end processes that store files on the IPFS and log the file's CID on the blockchain. During the approval process, the front end allows clients to call functions of the smart contract (approve or reject) which changes the status of the project and releases funds if the conditions of the contract are met. Lastly, the front end gets the data and displays it for the sake of transparency on both the back end and the blockchain. The diagram showcases the front end's importance in managing asynchronous operations, integrating wallet functionalities and managing real-time user interface changes based on the response of the distributed systems.



4.1.3. Use case diagram

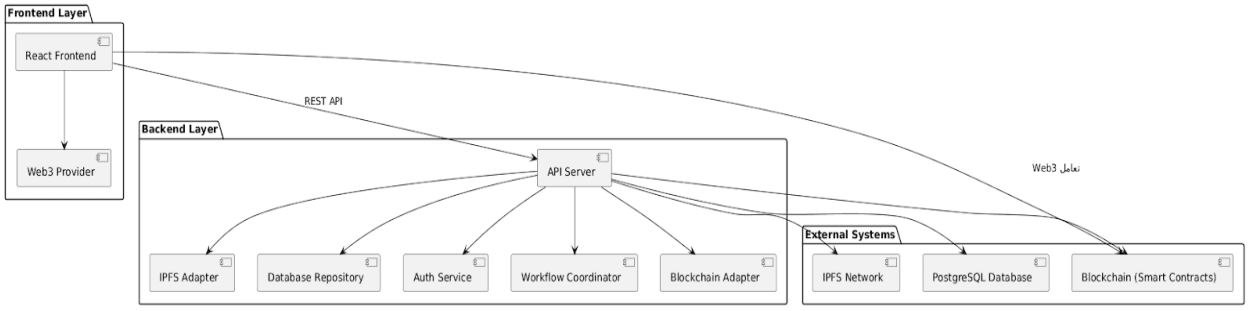
The use case diagram highlights the system's functional needs across different roles, namely clients, freelancers, and admins. All roles interface with the system to accomplish project assignments, approve milestones, submit deliverables, and resolve disputes. A majority of these activities are dependent on other activities such as cryptographic authentication, file uploads on the Interplanetary File Service (IPFS), and validation of the state of the blockchain. For frontend development, this diagram is essential as it specifies the corresponding UIs and the flows of interactions. It also directs the development of role-oriented dashboards, forms, and components, and the implementation of role-based access control (RBAC) modules. Additionally, it ensures that the frontend properly orchestrates the sequence of user actions, and that there is adequate integration with backend and blockchain services to complete each use case.



4.1.4. System Architecture

The system is designed in layers and modules. The front end serves as the orchestration layer through user interaction. The front end, built on React, connects to the back end through a REST

API, and communicates with the blockchain as a Web3 provider for wallet and transaction signing responsibilities. The back end functions as the intermediary layer, handling the business logic and authentication, and manages the orchestration of the other layers. For decentralized storage, the back end communicates with IPFS to receive CIDs, which are later used to fetch files. The back end also manages communication with the blockchain through a smart contracts adapter to control project states, approve milestones, and manage escrow contract payments. For structured information like user, project, and milestone details, a relational database is used. For the front end, the architecture involves the management of a practical application state, asynchronous API calls, Web3 calls, a relational database and blockchain off (or on) chain data to keep the UI in sync, and asynchronous API calls.



Appendix

• App.js: Root components take care of global state, wallet connection, session management with toast notifications, route definitions.

• Landing.js: This is the public landing page with hero section, features grid, and call-to-action.

• Login.js: An authentication page that can switch between MetaMask wallet connection and email/password login through tabs.

• FreelancerDashboard.js: Freelancers' full portal; here you have tab navigations of Dashboard, Find Jobs, My Contracts, My Profile, Messages.

• ClientDashboard.js: Full portal for the client; tab navigations of Dashboard, Explore Jobs, My Contracts, Messages.

• FreelancerProfile.js: Profile-managing view to edit skill tags, bio, hourly rate, and portfolio links.

• ExploreJobs.js: The job browsing front, including filters, search options, and submission of application for clients.

• MyContracts.js: List and detail view of contracts and their tracking of milestones for both user roles.

• Messages.js: The conversation thread with shared messages that do work like normal chatting.

3.3.3 User Interaction (Frontend Direction)

1) wallets connect user Authentication Success.

2) User accesses dashboard - Projects.

3) User develops a project or applies.

4) Display the table of projects and milestones according to the system.

5) User has uploaded deliverables - Communicates with IPFS.

6) System reports the state of the project - Report confirmation.

References (APA Style)

Shahzad, A. (2024). Security vulnerabilities in freelancing platforms. Luleå University of Technology.

Kunwar, R., et al. (2024). Job matching systems in freelancing platforms. IARJSET.

Singh, P., et al. (2024). DLancer: A decentralized freelancing platform using blockchain. Nitte Meenakshi Institute of Technology.

Gupta, A., et al. (2020). Challenges in freelance software development. Processes (MDPI).

Tamsetwar, S., et al. (2026). Hybrid Web3 architecture for freelancing platforms. IRJMETS.

Fulker, D., & Riedl, C. (2024). Collaboration in gig economy platforms. ACM CSCW.

Empty

Kasenda, M., et al. (2024). The Role and Evolution of Frontend Developers in the SOftware Development Industry. Journal of Engineering and Applied Science, Springer.

Reuter, J., et al. (2022) Comparative Analysis on Various CSS and JavaScript Frameworks. International Research Journal of Modernization in Engineering Technology and Science (IRJMETS).